

Price & Catalog Sync (Near-real-time ETL)

Summary:

The main goal of this project is to automatically update product prices and catalog information across all company systems almost instantly after a change is made in the source system.

This project demonstrates a robust, scalable, and near-real-time data pipeline built on Amazon Web Services (AWS). The primary objective is to synchronize product price and catalog information across company systems, catering to two distinct business needs: live analytics and historical analysis.

To achieve this, a dual-pipeline architecture was implemented. The first pipeline provides low-latency data to an Amazon Redshift data warehouse for powering live dashboards. The second pipeline archives all raw data and builds a historical data lake in Amazon S3 using a Medallion (Bronze-Silver-Gold) Architecture and Slowly Changing Dimensions (SCD Type 2). The final, business-ready data is made available for querying through Amazon Athena.

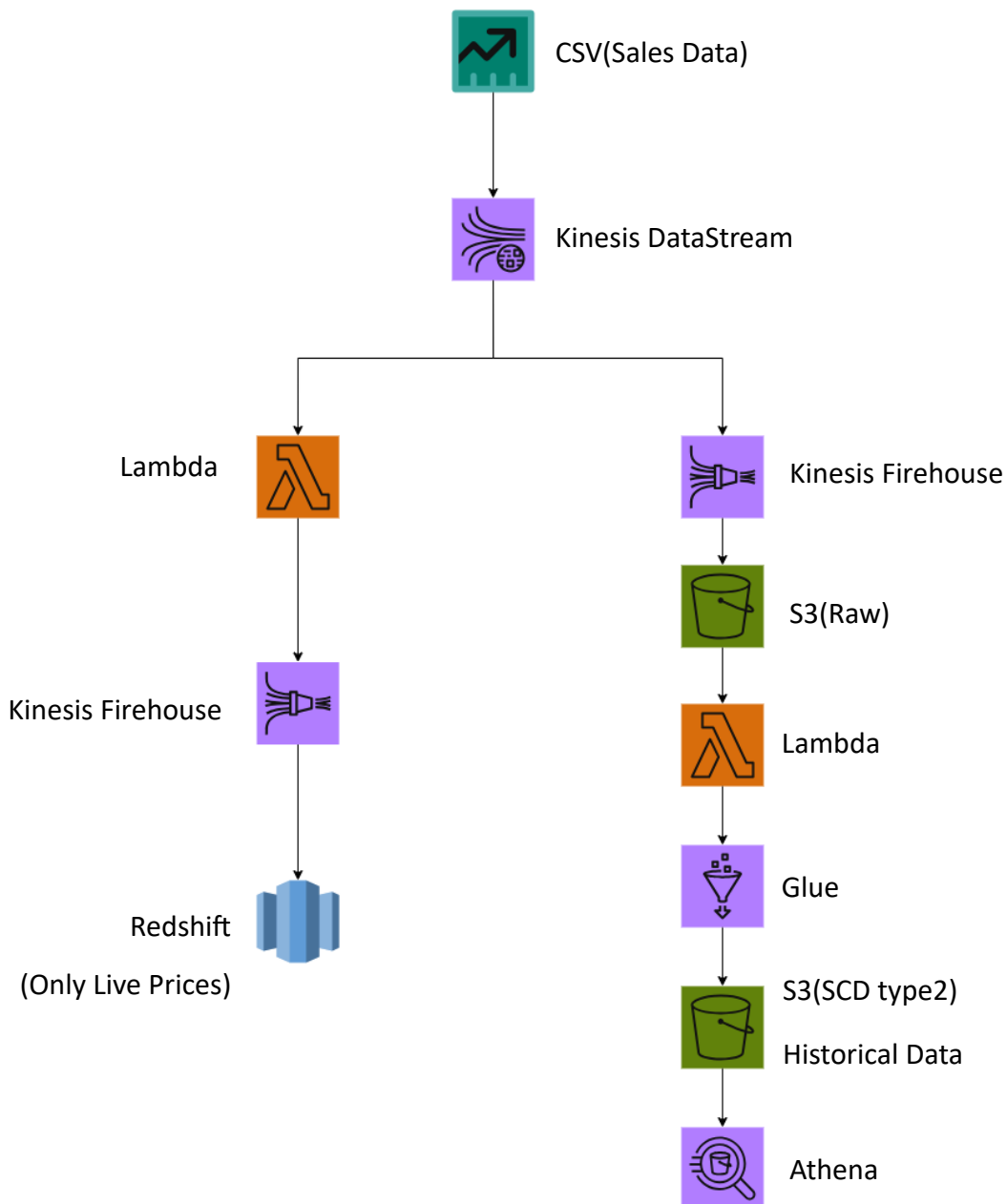
Technologies Used:

- Data Ingestion: AWS Kinesis Data Streams
 - Data Processing: AWS Lambda, AWS Glue, PySpark
 - Data Storage: Amazon S3, Amazon Redshift
 - Data Warehousing & Querying: Amazon Redshift, Amazon Athena
 - Data Format: Delta Lake
 - Languages/Libraries: Python, Boto3, Pandas, PySpark, SQL
-

Solution Architecture:

The architecture is designed to handle streaming data efficiently, splitting the stream into two parallel paths to serve different purposes without compromise.

- Pipeline 1: The Real-Time Dashboard Pipeline (Hot Path): This path is optimized for speed. Its goal is to get the most current data into a queryable system as fast as possible to power live applications.
- Pipeline 2: The Historical Data Lake Pipeline (Cold Path): This path is optimized for depth and reliability. Its goal is to create a permanent, detailed, and versioned historical record of all data for business intelligence and machine learning.



Data Pipeline Implementation

Pipeline 1: Real-Time Analytics

This pipeline ensures that live dashboards always have access to the latest product prices.

1. Data Producer: A Python script (producer.py) simulates a data source, reading product data from a CSV file and sending it as JSON records to a Kinesis Data Stream.

```
import pandas as pd
import boto3
import json
import time

# AWS credentials
aws_access_key = 'AKIAXDCBJYPYNKAHNCZ'
aws_secret_key = '9kpSaqYqsm3ihHbx+LwNkTagmDaxX1S1fJF652i'
region_name = 'ap-south-1'
stream_name = 'demo'

# Initialize Kinesis client
kinesis_client = boto3.client(
    'kinesis',
    region_name=region_name,
    aws_access_key_id=aws_access_key,
    aws_secret_access_key=aws_secret_key
)

# Read Local CSV
df = pd.read_csv("dataset.csv")

# Ensure numeric fields are consistent (float for price)
df['price'] = df['price'].astype(float)

# Convert to list of dicts (JSON objects)
records = df.to_dict(orient='records')

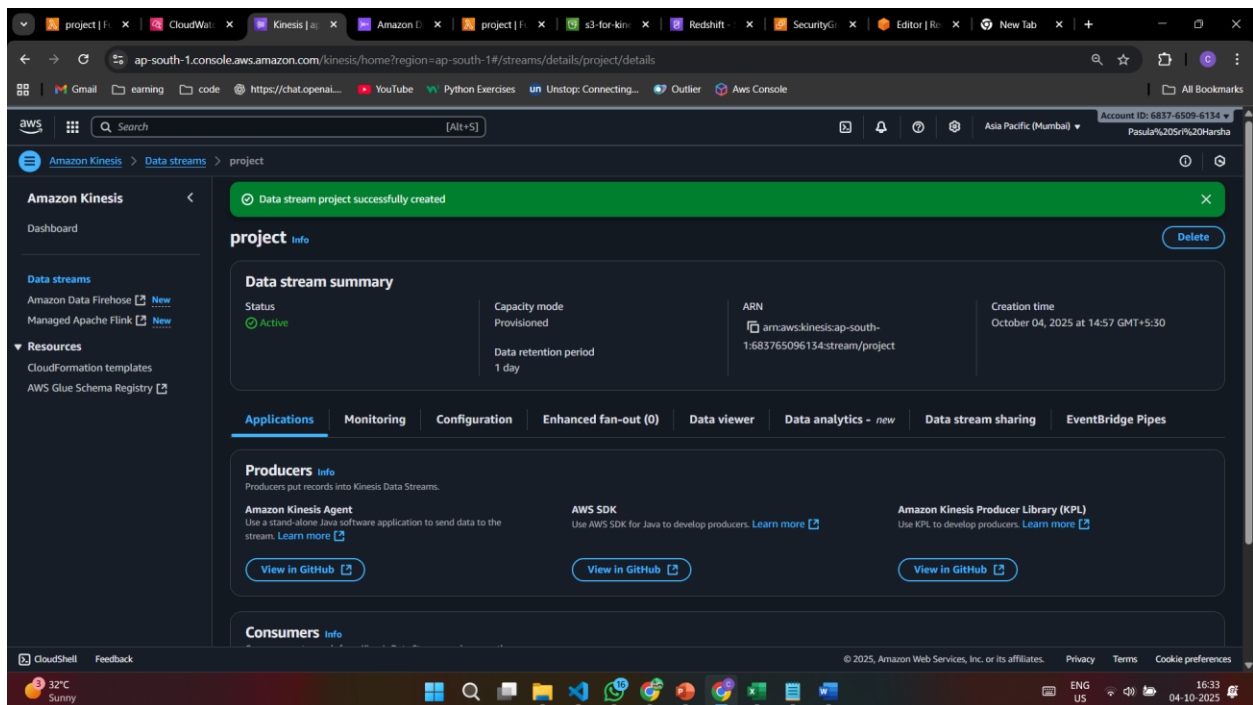
# Send all records as a single JSON array
payload = json.dumps(records)
kinesis_client.put_record(
    StreamName=stream_name,
    Data=payload.encode('utf-8'),
    PartitionKey="partition-1"
)

print("✅ Sent payload to Kinesis:")
print(json.dumps(records, indent=2))
```

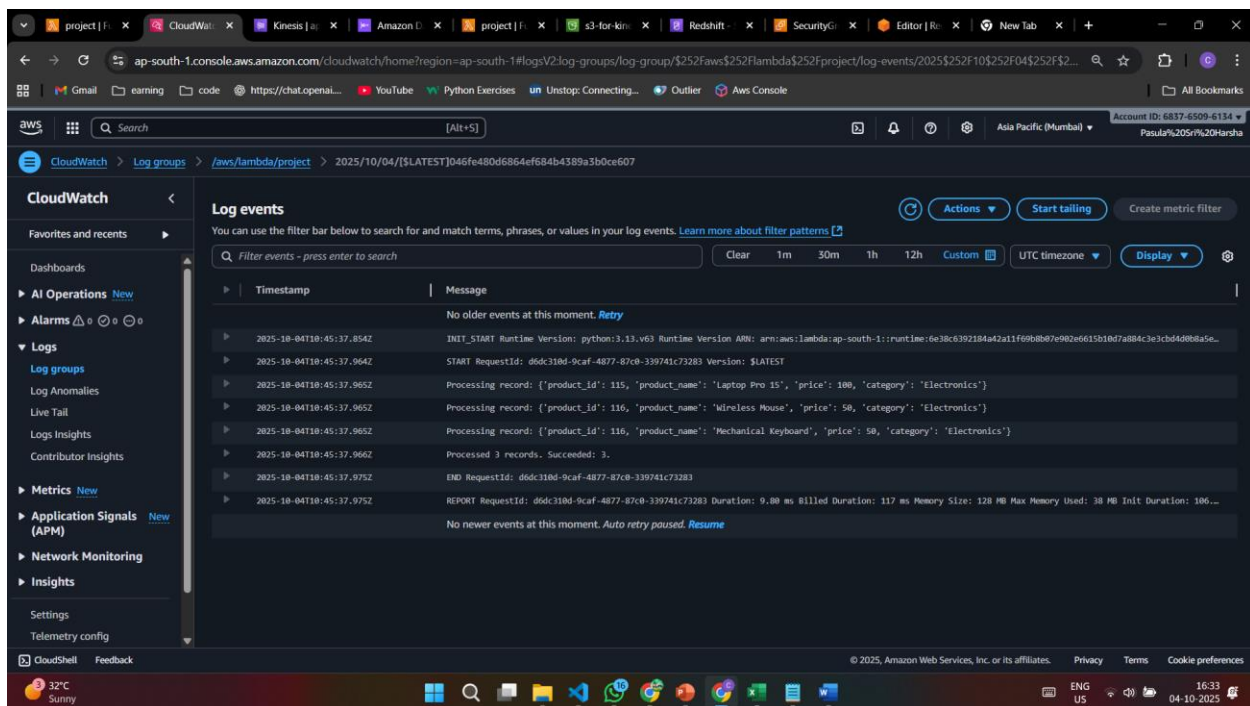
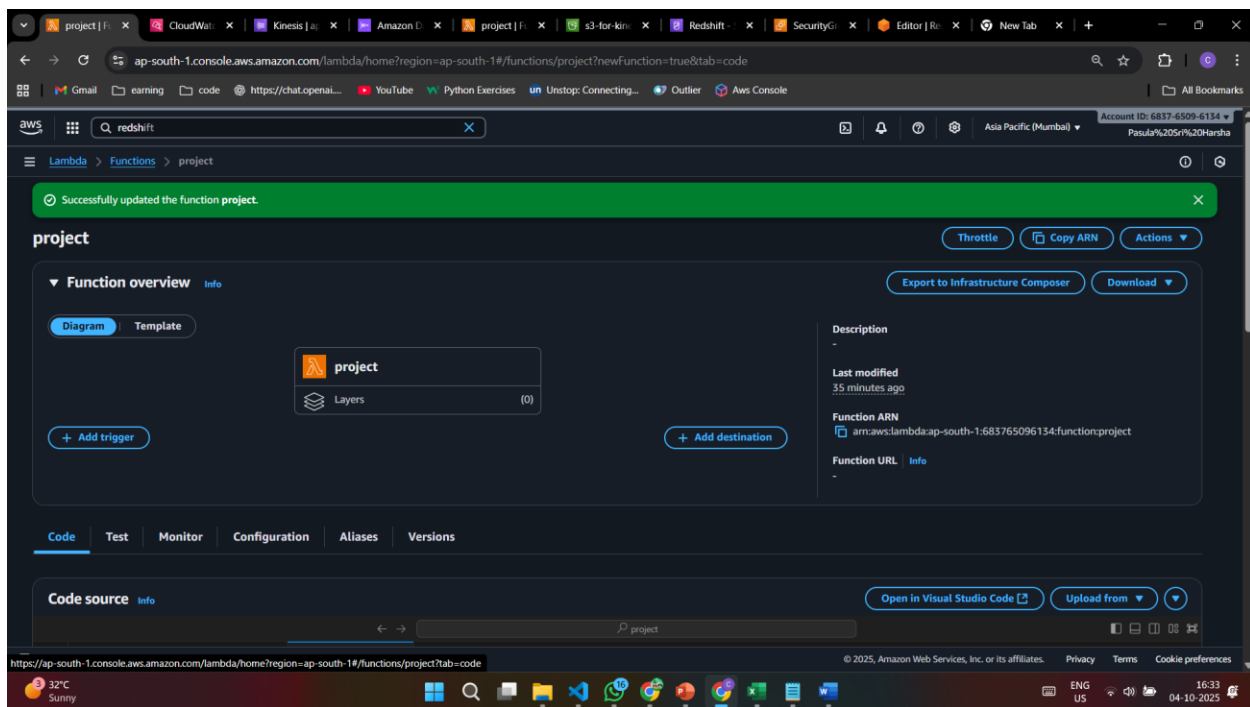
The screenshot shows a Visual Studio Code editor with a Python script named `updated_main.py`. The script imports `pandas`, `boto3`, `json`, and `time`. It defines AWS credentials and a Kinesis client. It reads a local CSV file `dataset.csv`, ensures numeric fields are consistent, converts the data to a list of dictionaries, and sends the records to a Kinesis stream named `demo` in the `ap-south-1` region. The terminal shows the command prompt at `PS C:\Users\ADMIN\Desktop\DETraining\AWS Data Engineering\Projects>`.

```
1 import pandas as pd
2 import boto3
3 import json
4 import time
5
6 # AWS credentials
7 aws_access_key = 'AKIAQCB37YPYKANNICZ'
8 aws_secret_key = '9hpSa9Ysq3iHf8x+LwKTagmDaxXISlFjF652i'
9 region_name = 'ap-south-1'
10 stream_name = 'demo'
11
12 # Initialize Kinesis client
13 kinesis_client = boto3.client(
14     'kinesis',
15     region_name=region_name,
16     aws_access_key_id=aws_access_key,
17     aws_secret_access_key=aws_secret_key
18 )
19
20 # Read local CSV
21 df = pd.read_csv("dataset.csv")
22
23 # Ensure numeric fields are consistent (float for price)
24 df['price'] = df['price'].astype(float)
25
26 # Convert to list of dicts (JSON objects)
27 records = df.to_dict(orient='records')
28
29 # Send all records as a single JSON array
30 payload = json.dumps(records)
31 kinesis_client.put_record(
32     StreamName=stream_name,
33     Data=payload.encode('utf-8'),
34     PartitionKey="partition-1"
```

2. Ingestion: Amazon Kinesis Data Streams ingests the streaming data, providing a scalable and durable buffer.



3. Transformation: An AWS Lambda function is triggered by the Kinesis stream. It performs quick, essential transformations and cleaning on the fly.



4. Loading: The transformed data is sent to Kinesis Data Firehose, which efficiently batches and loads the data into an Amazon Redshift table.

project | F... x CloudWat... x Kinesis | ap... x Amazon D... x project | F... x s3-for-kine... x Redshift -... x SecurityG... x Editor | Re... x New Tab x +

ap-south-1.console.aws.amazon.com/firehose/home?region=ap-south-1#/details/KDS-RED-op2yy/destination-logs

Amazon Data Firehose > Firehose streams > KDS-RED-op2yy

Amazon Data Firehose

Firehose streams

▼ Resources

- What's new
- Developer guide
- API reference

KDS-RED-op2yy

Delete Firehose stream

Firehose stream details

Status Active	Destination Amazon Redshift	Data transformation Enabled	Creation time October 04, 2025 at 15:19 GMT+5:30
Source Amazon Kinesis Data Streams	ARN arn:aws:firehose:ap-south-1:583765096134:deliverystream/KDS-RED-op2yy		Error logs status 11 Destination error logs

► Test with demo data

Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration **Destination error logs**

Destination error logs details

Log group in Amazon CloudWatch Logs
/aws/kinesisfirehose/KDS-RED-op2yy

Log stream
DestinationDelivery

Destination error logs (11)

project | F... x CloudWat... x Kinesis | ap... x Amazon D... x s3-for-kine... x Redshift - Se... x SecurityGrou... x Editor | Red... x New Tab x +

ap-south-1.console.aws.amazon.com/s3/buckets/s3-for-kinesisdatafirehouse?region=ap-south-1&bucketType=general

Amazon S3 > Buckets > s3-for-kinesisdatafirehouse

s3-for-kinesisdatafirehouse

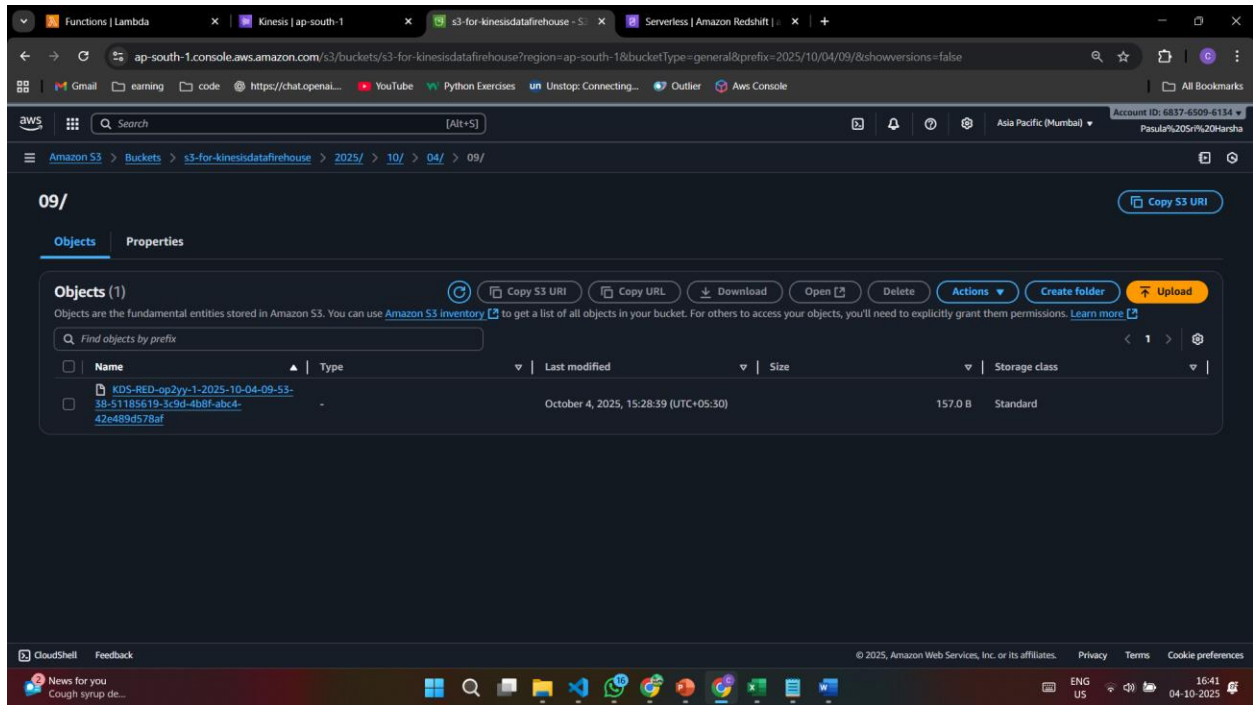
Objects Properties Permissions Metrics Management Access Points

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
2025/	Folder	-	-	-
manifests/	Folder	-	-	-



5. Warehousing: Amazon Redshift stores the current prices. A MERGE logic is periodically run to update the `product_prices_live` table, ensuring it only contains the single, most recent price for each product.

A table must be created in the Redshift cluster before Firehose can load data into it.

```
CREATE TABLE live_prices ( product_id VARCHAR(255) NOT NULL, price DECIMAL(10, 2) NOT NULL, event_timestamp TIMESTAMP NOT NULL );
```

project | Fun...CloudWatch | Kinesis | ap-...Amazon Data...s3-for-kines...Serverless | ^...SecurityGrou...Editor | Redo...New tab | +

ap-south-1.console.aws.amazon.com/redshiftv2/home?region=ap-south-1#/serverless-dashboard

Gmail | eaming | code | https://chat.openai... | YouTube | Python Exercises | Unstop: Connecting... | Outlier | Aws Console | All Bookmarks

AWS

Search [Alt+S]

Asia Pacific (Mumbai) Account ID: 6837-6509-6134 Pasula%20Sr%20Harsha

Amazon Redshift Serverless

Serverless dashboard

Query data Create workgroup

Namespace overview

Filter namespace All namespaces

Namespace data from your account

Total snapshots0

Datashares in my account0

Datashares requiring authorization0

Datashares from other accounts0

Datashares requiring association0

Namespaces / Workgroups

Namespace	Status	Workgroup	Status	Average query duration	Average number of queries run daily
default-namespace	Available	default-workgroup	Available		

Total compute usage - new

Choose a workgroup Last hour

To visualize the costs of your total compute usage, go to AWS Cost Explorer

Total consumed RPU hours

Retrieving a workgroup's total compute usage uses the workgroup's compute capacity, and might incur associated costs.

Queries metrics

default-workgroup Last hour All running and queued queries View details

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Sunny

16:34 04-10-2023

project | Fun... x CloudWatch | x Kinesis | ap-... x Amazon Data... x s3-for-kines... x Redshift - Se... x SecurityGrou... x Editor | Red... x New Tab x +

ap-south-1.console.aws.amazon.com/redshiftv2/home?region=ap-south-1#/serverless-workgroup/workgroup=default-workgroup

Search [Alt+S]

General information

Workgroup default-workgroup	Date created October 04, 2025, 15:05 (UTC+05:30)	Endpoint default-workgroup.683765096134.ap-south-1.redshift-serverless.amazonaws.com:5439/dev
Namespace default-namespace	Base capacity 4 RPU's	JDBC URL jdbc:redshift://default-workgroup.683765096134.ap-south-1.redshift-serverless.amazonaws.com:5439/dev
Workgroup ARN arn:aws:redshift-serverless:ap-south-1:683765096134:workgroup/6f48155f-481e-4c06-8880-cc785f0177bf	Custom domain name -	ODBC URL Driver=(Amazon Redshift (x64)); Server=default-workgroup.683765096134.ap-south-1.redshift-serverless.amazonaws.com; Database=dev
Workgroup version 1.0.127211	Patch version Patch 193	
Status Available	Track - new Current	

Base capacity is now in the Performance and cost controls under the Performance tab, where you can choose between setting a base capacity in RPU's or a price-performance target.

Publicly accessible turned on. Publicly accessible for this Redshift instance turned on. This means this instance is now accessible to the public internet. Any external sources will be able to connect and access the data within this Redshift cluster. [Learn more](#)

Query and database monitoring Data access Limits Performance Tags

CloudShell Feedback

32°C Sunny

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

project | Fun... x CloudWatch | x Kinesis | ap-... x Amazon Data... x s3-for-kines... x Redshift - Se... x EC2 | ap-sou... x Editor | Red... x New Tab x +

ap-south-1.console.aws.amazon.com/redshiftv2/home?region=ap-south-1#/namespace/default-namespace

Search [Alt+S]

You can now register your Redshift table definitions to AWS Glue Data Catalog, making them accessible from other query engines such as EMR Serverless, Athena and other Redshift data warehouses. To get started registering a namespace, select a Redshift serverless namespace from the namespaces, then select "Register with AWS Glue Data Catalog" from the Actions menu. [Learn more](#)

Amazon Redshift Serverless > Namespace configuration > default-namespace

default-namespace [Info](#)

General information

Namespace default-namespace	Status Available	Admin user name admin
Namespace ID 23c6ef1c-86b3-4348-a304-de2e3b517f6c	Date created October 04, 2025, 15:05 (UTC+05:30)	Database name dev
Namespace ARN arn:aws:redshift-serverless:ap-south-1:683765096134:namespace/23c6ef1c-86b3-4348-a304-de2e3b517f6c	Storage used 113 MB	Total table count 2
Namespace register status Deregistered		

Workgroup Data backup Database Security and encryption Datashares Zero-ETL integrations Resource policy Tags

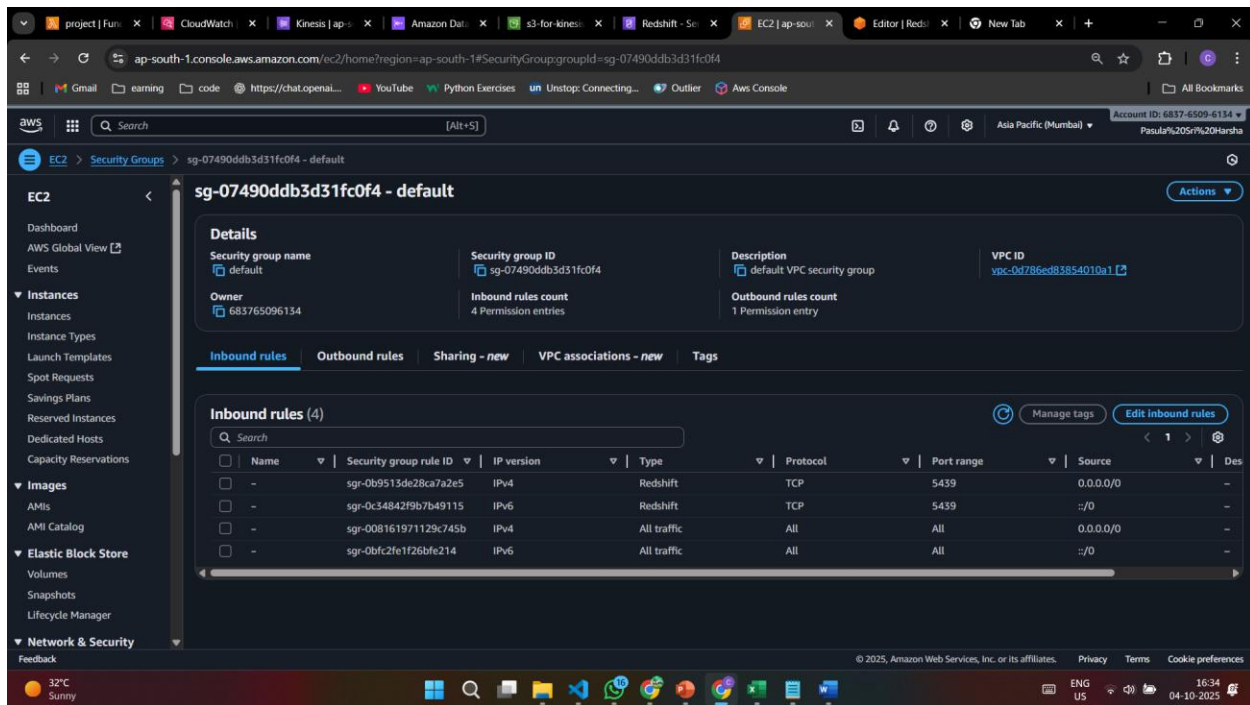
Workgroup name

Actions

CloudShell Feedback

32°C Sunny

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Redshift Query Editor:

```
-- Create your final "live" table once (same structure as your current table)
CREATE TABLE IF NOT EXISTS public.product_prices_live (
    product_id          INTEGER,
    product_name         VARCHAR(255),
    price                DECIMAL(10, 2),
    category             VARCHAR(100),
    last_updated_timestamp TIMESTAMPTZ
);

--- RUN THE FOLLOWING MERGE LOGIC PERIODICALLY ---

-- Step 1: Start a transaction to ensure safety
BEGIN;

-- Step 2: Delete any products from the final table that have new updates in the staging table.
DELETE FROM public.product_prices_live
USING public.product_prices
WHERE public.product_prices_live.product_id = public.product_prices.product_id;

-- Step 3: Insert only the absolute latest price for each product from the staging table into the
final table.
INSERT INTO public.product_prices_live
SELECT product_id, product_name, price, category, last_updated_timestamp
FROM (
    SELECT *,
           ROW_NUMBER() OVER(PARTITION BY product_id ORDER BY last_updated_timestamp DESC) as rn
    FROM public.product_prices
)
WHERE rn = 1;

-- Step 4 (Optional but recommended): Clear out the staging table so it's ready for the next batch.
TRUNCATE public.product_prices;

-- Step 5: Commit the transaction
END;
```

project | Fun | CloudWatch | Kinesis | ap- Amazon Data | s3-for-kines Redshift - Se EC2 | ap-sou Editor | Redi New Tab

ap-south-1.console.aws.amazon.com/sqlworkbench/home/region=ap-south-1#/client

Search [Alt+S]

Redshift query editor v2

Create Load data

Filter resources

Serverless: default-workgroup

native databases (2)

dev

public

Tables

product_prices

product_prices_...

product_prices

Distribution key: product_id

Sort keys: last_updated_timestamp

Sort type: Compound

Field Type NL

product_id integer NUL

product_name character varying(255) NUL

price numeric(10,2) NUL

category character varying(100) NUL

```
1 -- Create your final "live" table once (same structure as your current table)
2 CREATE TABLE IF NOT EXISTS public.product_prices_live (
3   product_id      INTEGER,
4   product_name    VARCHAR(255),
5   price           DECIMAL(10, 2),
6   category        VARCHAR(100),
7   last_updated_timestamp TIMESTAMPTZ
8 );
9
10 --- RUN THE FOLLOWING MERGE LOGIC PERIODICALLY ---
11
12 -- Step 1: Start a transaction to ensure safety
13 BEGIN;
```

Result 1 Result 2 Result 3 Result 4 Result 5 Result 6

Summary

Info:

- Relation "product_prices_live" already exists and will be skipped

Returned rows: 0

Query ID: 406267

Elapsed time: 20ms

Result set query:

/* RQE-V2 - RUN /pgproc */

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Sunny

project | Fun | CloudWatch | Kinesis | ap- Amazon Data | s3-for-kines Redshift - Se EC2 | ap-sou Editor | Redi New Tab

ap-south-1.console.aws.amazon.com/sqlworkbench/home/region=ap-south-1#/client

Search [Alt+S]

Redshift query editor v2

Create Load data

Filter resources

Serverless: default-workgroup

native databases (2)

dev

public

Tables

product_prices

product_prices_...

product_prices

Distribution key: product_id

Sort keys: last_updated_timestamp

Sort type: Compound

Field Type NL

product_id integer NUL

product_name character varying(255) NUL

price numeric(10,2) NUL

category character varying(100) NUL

```
1 SELECT
2
3 FROM
4 "dev"."public"."product_prices_live";
```

Result 1 (2)

product_id	product_name	price	category	last_updated_timesta...
115	Laptop Pro 15	100	Electronics	2025-10-04 10:45:37+00
116	Wireless Mouse	50	Electronics	2025-10-04 10:45:37+00

Query ID 406937 Elapsed time: 125 ms Total rows: 2

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Sunny

Pipeline 2: Historical Data Lake (Medallion Architecture)

This pipeline builds a reliable and queryable historical data lake.

1. Data Archiving: A second Kinesis Data Firehose subscribes to the same Kinesis Data Stream. It archives every raw, untouched record into an S3 Raw Zone, ensuring no data is ever lost.

The image displays two screenshots related to the Pipeline 2: Historical Data Lake (Medallion Architecture).

The top screenshot shows a VS Code editor with a Python script named `updated_main.py`. The script is designed to read data from a CSV file and send it to a Kinesis stream. Key components of the script include:

- Configuration for region and credentials.
- Initialization of the Kinesis client.
- Reading and preparing data from a CSV file.
- Conversion of the data into a list of dictionaries.
- Sending data to the Kinesis stream in a loop, with each record converted to a JSON string.
- Use of a dynamic partition key to distribute data across shards.

The bottom screenshot shows the AWS Kinesis console. A notification at the top states: "Data stream demo successfully created". The console displays the details for the data stream named `demo`, including its status (Active), capacity mode (Provisioned), and creation time (October 06, 2025 at 18:10 GMT+5:30). The console also provides links to various applications, producers, and consumers for the data stream.

ap-south-1.console.aws.amazon.com/firehose/home?region=ap-south-1#/details/KDS-S3-TNVCp/monitoring

Amazon Data Firehose > Firehose streams > KDS-S3-TNVCp

KDS-S3-TNVCp was successfully created.

KDS-S3-TNVCp

Firehose stream details

Status Active	Destination Amazon S3	Data transformation Not enabled	Creation time October 06, 2025 at 18:12 GMT+5:30
Source Amazon Kinesis Data Streams	ARN arn:aws:firehose:ap-south-1:487615743519:deliverystream/KDS-S3-TNVCp	Dynamic partitioning Not enabled	Error logs status Destination error logs

Test with demo data
Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration Destination error logs

Firehose stream metrics

Investigate with AI - new 1h 3h 12h 1d 3d 1w Custom UTC timezone Explore related

Records read from Kinesis Data Streams (Sum)	Bytes read from Kinesis Data Streams (Sum)	DescribeStream operations throttled (Average)
Count 1	Bytes 1	Count 1
0.0	0.0	0.0

ap-south-1.console.aws.amazon.com/firehose/home?region=ap-south-1#/details/KDS-S3-TNVCp/edit-destination

Amazon Data Firehose > Firehose streams > KDS-S3-TNVCp > Edit destination settings

KDS-S3-TNVCp was successfully created.

Edit destination settings

Destination settings
Specify the destination settings for your Firehose stream.

S3 bucket
s3://project-raw-ncpl Browse Create

Format: s3://bucket

New line delimiter
You can configure your Firehose stream to add a new line delimiter between records in objects that are delivered to Amazon S3.
☐ Not enabled
☒ Enabled

Dynamic partitioning
Dynamic partitioning enables you to create targeted data sets by partitioning streaming S3 data based on partitioning keys. You can partition your source data with inline parsing and/or the specified AWS Lambda function. You can enable dynamic partitioning only when you create a new Firehose stream. You cannot enable dynamic partitioning for an existing Firehose stream. Enabling dynamic partitioning incurs additional costs per GB of partitioned data. For more information, see Amazon Data Firehose pricing.
☒ Not enabled
☐ Enabled

S3 bucket prefix - optional
By default, Amazon Data Firehose appends the prefix 'YYYY/MM/DD/' (in UTC) to the data it delivers to Amazon S3. You can override this default by specifying a custom prefix that includes expressions that are evaluated at runtime.
Enter a prefix

You can repeat the same keys in your S3 bucket prefix. Maximum S3 bucket prefix characters: 1024.

ap-south-1.console.aws.amazon.com/firehose/home?region=ap-south-1#/details/KDS-S3-TNVCp/edit-destination

UTC

▼ Buffer hints, compression, file extension and encryption
The fields below are pre-populated with the recommended default values for S3. Pricing may vary depending on storage and request costs.

S3 buffer hints
Amazon Data Firehose buffers incoming records before delivering them to your S3 bucket. Record delivery is triggered once the value of either of the specified buffering hints is reached.

Buffer size
The higher buffer size may be lower in cost with higher latency. The lower buffer size will be faster in delivery with higher cost and less latency.
5 MIB
Minimum: 1 MIB, maximum: 128 MIB, Recommended: 5 MIB.

Buffer interval
The higher interval allows more time to collect data and the size of data may be bigger. The lower interval sends the data more frequently and may be more advantageous when looking at shorter cycles of data activity.
60 seconds
Minimum: 0 seconds, maximum: 900 seconds, Recommended: 300 seconds.

Compression for data records
Amazon Data Firehose can compress records before delivering them to your S3 bucket.
☒ Not enabled
☐ GZIP
☐ Snappy
☐ Zip
☐ Hadoop-Compatible Snappy

File extension format - optional
Enter the full extension format. If you want to add ".gzip" file extension for your ".json" files, enter ".json.gz".
Enter a file extension (for example, ".json", ".zip", ".json.gz")
File extension must start with a period and may only contain allowed characters: 0-9a-z, "-", ".", "_". File extension cannot exceed 128 characters.

ap-south-1.console.aws.amazon.com/s3/buckets?region=ap-south-1&bucketType=general

General purpose buckets **Directory buckets**

General purpose buckets (5) [info](#)
Buckets are containers for data stored in S3.

Name	AWS Region	Creation date
aws-glue-assets-487615743519-ap-south-1	Asia Pacific (Mumbai) ap-south-1	October 6, 2025, 11:47:03 (UTC+05:30)
project-medallion-ncpl	Asia Pacific (Mumbai) ap-south-1	October 5, 2025, 23:42:03 (UTC+05:30)
project-raw-ncpl	Asia Pacific (Mumbai) ap-south-1	October 6, 2025, 18:17:18 (UTC+05:30)
raw-s3-project-historical-price-data	Asia Pacific (Mumbai) ap-south-1	October 5, 2025, 18:03:49 (UTC+05:30)
s3-project-historical-price-data-1	Asia Pacific (Mumbai) ap-south-1	October 5, 2025, 15:57:00 (UTC+05:30)

Account snapshot [info](#)
[Updated daily](#)
Storage Lens provides visibility into storage usage and activity trends. [View dashboard](#)

External access summary - new [info](#)
[Updated daily](#)
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

2. ETL Trigger: An AWS Lambda function is configured with an S3 trigger. When a new raw data file lands in the S3 Raw Zone, this function automatically starts an AWS Glue job for processing.

```
# Lambda_function.py

import boto3
import os
import urllib.parse
import json

s3 = boto3.client('s3')
glue_client = boto3.client('glue')
GLUE_JOB_NAME = os.environ.get('GLUE_JOB_NAME')

def lambda_handler(event, context):
    if not GLUE_JOB_NAME:
        raise ValueError("Glue job name not configured (GLUE_JOB_NAME missing).")

    all_records = []

    for record in event.get('Records', []):
        bucket = record['s3']['bucket']['name']
        key = urllib.parse.unquote_plus(record['s3']['object']['key'], encoding='utf-8')

        # Read JSON from S3
        json_obj = s3.get_object(Bucket=bucket, Key=key)
        json_data = json_obj['Body'].read().decode('utf-8')

        # ===== MODIFIED CODE BLOCK START =====
        # Process the file line by line, as it's in JSON Lines format
        for line in json_data.splitlines():
            if not line.strip(): # Skip empty lines
                continue
            try:
                # Parse each individual line as a JSON object
                record_obj = json.loads(line)
                all_records.append(record_obj)
            except json.JSONDecodeError:
                print(f"⚠ Failed to parse a line in file s3://{bucket}/{key}. Skipping line: {line}")
        # ===== MODIFIED CODE BLOCK END =====

    if not all_records:
        print("No valid JSON data found in provided files.")
        return {'statusCode': 200, 'body': 'No data to process'}

    # Create a single JSON array file
    output_bucket = bucket # reuse same bucket or change as needed
    output_key = f"processed/combined_data_{context.aws_request_id}.json" # Use a unique name
    s3.put_object(
        Bucket=output_bucket,
        Key=output_key,
        Body=json.dumps(all_records, indent=2).encode('utf-8')
    )

    print(f"✅ Combined JSON written to s3://{output_bucket}/{output_key}")

    # Start Glue job ONCE for all data
    response = glue_client.start_job_run(
        JobName=GLUE_JOB_NAME,
        Arguments={
```

```

        '--s3_file_paths_list': f"s3://{output_bucket}/{output_key}"
    }
)

job_run_id = response['JobRunId']
print(f"✅ Glue job triggered successfully: {job_run_id}")

return {
    'statusCode': 200,
    'body': json.dumps(f"Started Glue job {job_run_id}")
}

```

The screenshot displays the AWS Lambda console interface for a function named 'demo-lambda'. The top navigation bar shows the AWS logo, search bar, and account information (Asia Pacific (Mumbai), Yashwanth).

The main content area is divided into two tabs: 'Function overview' and 'Code source'. The 'Function overview' tab is active, showing a diagram of the function's architecture. It includes a 'demo-lambda' function icon, a 'Layers' section with '(0)' layers, and an 'S3' trigger. A '+ Add trigger' button is visible. To the right, a 'Description' panel shows 'Last modified 2 hours ago', 'Function ARN: arn:aws:lambda:ap-south-1:487615743519:function:demo-lambda', and 'Function URL: Info'.

The 'Code source' tab is also visible, showing a file explorer with 'lambda_function.py' and its contents. The code includes imports for boto3, os, urllib.parse, and json.

The bottom of the screen shows a Windows taskbar with various application icons and a system clock indicating 20:29 on 06-10-2023.

ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/demo-lambda/subtab=envVars&tab=configure

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

aws Search [Alt+S] Account ID: 4876-1574-3519 Asia Pacific (Mumbai) Yashwanth

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Lambda > Functions > demo-lambda

Diagram Template

demo-lambda

Layers (0)

+ Add trigger

+ Add destination

Description

Last modified 2 hours ago

Function ARN arn:aws:lambda:ap-south-1:487615743519:function:demo-lambda

Function URL info

Code Test Monitor Configuration Aliases Versions

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

VPC

RDS databases

Monitoring and operations tools

Environment variables (1)

The environment variables below are encrypted at rest with the default Lambda service key.

Find environment variables

Key	Value
GLUE_JOB_NAME	glue-demo

Edit

https://ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/demo-lambda?tab=configure

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

28°C Mostly cloudy

ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/demo-lambda/edit/relation?subtab=triggers&tab=configure

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

aws Search [Alt+S] Account ID: 4876-1574-3519 Asia Pacific (Mumbai) Yashwanth

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Lambda > Functions > demo-lambda > Edit trigger

Edit trigger

Trigger configuration

S3

aws asynchronous storage

Bucket

Choose or enter the ARN of an S3 bucket that serves as the event source. The bucket must be in the same region as the function.

arn:aws:s3::project-raw-mcpl

Bucket must be in region ap-south-1

Event types

Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.

All object create events

Prefix - optional

Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any special characters must be URL encoded.

2025/

Suffix - optional

Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any special characters must be URL encoded.

e.g. .jpg

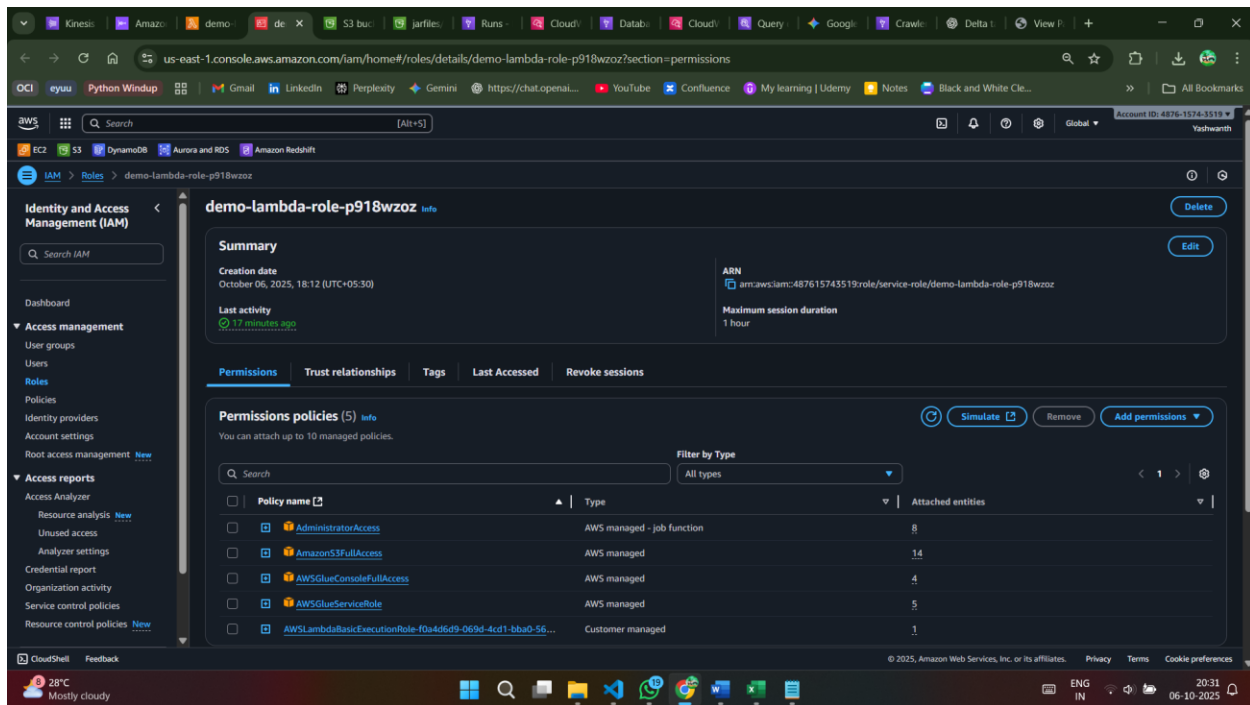
Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. Learn more about the Lambda permissions model.

Cancel Save

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

28°C Mostly cloudy



3. Heavy ETL with AWS Glue & PySpark: A sophisticated AWS Glue job, written in PySpark, performs the core data transformation using the Medallion Architecture and Delta Lake format.
 - Bronze Layer: Raw data from S3 is loaded into a Bronze Delta table. This layer holds the unmodified source data with a consistent schema.

```
# glue_scd_job.py
# UPDATED SCRIPT (v4.6) - Fixed DELTA_FAILED_TO_MERGE_FIELDS for 'price'
# Key Changes:
# - Added schema overwrite mode for Bronze table initialization
# - Enhanced error handling with schema mismatch detection
# - Added option to evolve schema or overwrite based on compatibility

import sys
from aws glue. utils import getResolvedOptions
from pyspark. context import SparkContext
from aws glue. context import GlueContext
from aws glue. job import Job
from delta. tables import DeltaTable
from pyspark. sql. functions import col, lit, trim, current_timestamp, coalesce
from pyspark. sql. types import StringType, DoubleType, LongType, TimestampType, StructType, StructField
from pyspark. sql. utils import AnalysisException

# --- Spark Session and Glue Context Initialization ---
sc = SparkContext. getOrCreate()
glueContext = GlueContext(sc)
spark = glueContext. spark_session

# Enable Delta configurations
spark. conf. set("spark. databricks. delta. schema. autoMerge. enabled", "true")
spark. conf. set("spark. sql. legacy. decimal. retainFractionDigitsOnTruncate", "true")
# Allow type widening for better compatibility (e.g., Int -> Long, Float -> Double)
```

```

spark.conf.set("spark.databricks.delta.schema.typeEvolution.enabled", "true")

job = Job(glueContext)

# --- Argument Parsing ---
args = getResolvedOptions(sys.argv, ['JOB_NAME', 's3_file_paths_list'])
job.init(args['JOB_NAME'], args)

# --- Parameters ---
all_new_paths_str = args['s3_file_paths_list']
s3_paths_to_process = all_new_paths_str.split(',') if all_new_paths_str else []

processed_data_bucket = "s3://project-medallion-ncpl/"
bronze_path = f"{processed_data_bucket}bronze/prices/"
silver_path = f"{processed_data_bucket}silver/prices/"
gold_path = f"{processed_data_bucket}gold/prices_history/"

# --- Graceful Exit if No Files ---
if not s3_paths_to_process:
    print("No new S3 file paths provided. Job is finishing.")
    job.commit()
    sys.exit(0)

print(f"Received {len(s3_paths_to_process)} file(s) to process.")
print(f"Bronze path: {bronze_path}")
print(f"Silver path: {silver_path}")
print(f"Gold path: {gold_path}")

# --- BRONZE LAYER (RAW INGESTION) ---
# Define consistent schema with DoubleType for price
bronze_schema = StructType([
    StructField("product_id", StringType(), True),
    StructField("product_name", StringType(), True),
    StructField("price", DoubleType(), True), # Consistent DoubleType
    StructField("category", StringType(), True),
    StructField("supplier_timestamp", StringType(), True)
])

try:
    raw_updates_df = spark.read \
        .schema(bronze_schema) \
        .option("multiline", "true") \
        .json(s3_paths_to_process)

    if raw_updates_df.rdd.isEmpty():
        print("Input files were empty or contained no valid JSON. Job finished.")
        job.commit()
        sys.exit(0)

    print(f"Read {raw_updates_df.count()} new records from raw source.")
    print("Schema of incoming data:")
    raw_updates_df.printSchema()
    raw_updates_df.show(5, truncate=False)

except AnalysisException as e:
    print(f"Error reading JSON data. Exiting. Error: {e}")
    job.commit()
    sys.exit(0)

# Write to Bronze with enhanced error handling
print("Writing new records to Bronze Delta table.")
try:
    bronze_exists = DeltaTable.isDeltaTable(spark, bronze_path)

    if bronze_exists:

```

```

# Check existing schema
existing_bronze = DeltaTable.forPath(spark, bronze_path)
existing_schema = existing_bronze.toDF().schema
print("Existing Bronze table schema:")
print(existing_schema)

# Try to detect price field type in existing table
price_field_existing = next((f for f in existing_schema.fields if f.name == "price"), None)
if price_field_existing:
    print(f"Existing 'price' field type: {price_field_existing.dataType}")

# Attempt append with mergeSchema
try:
    raw_updates_df.write.format("delta") \
        .option("mergeSchema", "true") \
        .mode("append") \
        .save(bronze_path)
    print("Bronze layer append successful with schema merge.")
except Exception as merge_error:
    print(f"Schema merge failed: {merge_error}")
    print("Attempting to overwrite Bronze table with correct schema...")

# Read all existing data
existing_data = existing_bronze.toDF()

# Cast existing data to match new schema
corrected_existing = existing_data.select(
    col("product_id").cast(StringType()),
    col("product_name").cast(StringType()),
    col("price").cast(DoubleType()), # Force to DoubleType
    col("category").cast(StringType()),
    col("supplier_timestamp").cast(StringType())
)

# Union with new data
combined_data = corrected_existing.union(raw_updates_df)

# Overwrite with corrected schema
combined_data.write.format("delta") \
    .option("overwriteSchema", "true") \
    .mode("overwrite") \
    .save(bronze_path)
print("Bronze table overwritten with corrected schema.")
else:
    # First time - create table
    print("Bronze table does not exist. Creating with correct schema.")
    raw_updates_df.write.format("delta") \
        .option("overwriteSchema", "true") \
        .mode("overwrite") \
        .save(bronze_path)
    print("Bronze layer created successfully.")

except Exception as e:
    print(f"Error writing to Bronze: {e}")
    import traceback
    traceback.print_exc()
    job.commit()
    sys.exit(1)

# --- SILVER LAYER (CLEANED & CURRENT STATE) ---
print("\n--- Starting Silver layer processing ---")

input_timestamp = coalesce(col("supplier_timestamp").cast(TimestampType()),
    current_timestamp()).alias("update_timestamp")

```



```

cleaned_updates_df = raw_updates_df \
    .withColumn("product_id", trim(col("product_id")).cast(LongType())) \
    .withColumn("price", col("price").cast(DoubleType())) \
    .withColumn("product_name", trim(col("product_name")).cast(StringType())) \
    .withColumn("category", trim(col("category")).cast(StringType())) \
    .withColumn("update_timestamp", input_timestamp) \
    .filter((col("product_id").isNotNull()) & (col("price") > 0)) \
    .dropDuplicates(["product_id"])

print(f"After cleaning, {cleaned_updates_df.count()} unique product updates are ready for Silver.")
cleaned_updates_df.show(5, truncate=False)

print("Upserting data into Silver Delta table...")
try:
    if DeltaTable.isDeltaTable(spark, silver_path):
        print("Silver table exists. Merging data.")
        silver_delta_table = DeltaTable.forPath(spark, silver_path)
        silver_delta_table.alias("target") \
            .merge(cleaned_updates_df.alias("source"), "target.product_id = source.product_id") \
            .whenMatchedUpdate(set={
                "price": "source.price",
                "product_name": "source.product_name",
                "category": "source.category",
                "update_timestamp": "source.update_timestamp"
            }) \
            .whenNotMatchedInsertAll() \
            .execute()
    else:
        print("Silver table does not exist. Creating it with the first batch.")
        cleaned_updates_df.write.format("delta") \
            .option("overwriteSchema", "true") \
            .save(silver_path)
        print("Silver layer operation complete.")
except Exception as e:
    print(f"Error in Silver merge: {e}")
    import traceback
    traceback.print_exc()
    job.commit()
    sys.exit(1)

# --- GOLD LAYER (SCD TYPE 2 PRICE HISTORY) ---
print("\n--- Starting Gold layer processing (SCD Type 2) ---")
source_df_for_gold = cleaned_updates_df

ALWAYS_INSERT_NEW_VERSION = False

try:
    if not DeltaTable.isDeltaTable(spark, gold_path):
        print("Gold table does not exist. Creating it with initial data.")
        initial_gold_df = source_df_for_gold \
            .withColumn("is_current", lit(True)) \
            .withColumn("start_date", col("update_timestamp")) \
            .withColumn("end_date", lit(None).cast(TimestampType()))
        initial_gold_df.write.format("delta") \
            .option("overwriteSchema", "true") \
            .save(gold_path)
    else:
        print("Gold table exists. Performing SCD Type 2 merge logic.")
        gold_delta_table = DeltaTable.forPath(spark, gold_path)

        current_history_df = gold_delta_table.toDF().where("is_current = true")

        joined_df = source_df_for_gold.alias("updates") \
            .join(current_history_df.alias("history"), "product_id", "left_outer") \
            .select(

```

```

        col("updates.*"),
        col("history.price").alias("history_price"),
        col("history.product_name").alias("history_product_name"),
        col("history.category").alias("history_category"),
        col("history.product_id").alias("history_product_id")
    )

    change_condition = (
        (col("price") != col("history_price")) |
        (col("product_name") != col("history_product_name")) |
        (col("category") != col("history_category")))
    )

    records_to_insert = joined_df.where(
        col("history_product_id").isNull() | change_condition | lit(ALWAYS_INSERT_NEW_VERSION)
    )

    keys_to_expire = records_to_insert \
        .where(col("history_product_id").isNotNull() & ~lit(ALWAYS_INSERT_NEW_VERSION)) \
        .select("product_id", "update_timestamp")

    if keys_to_expire.count() > 0:
        print(f"Found {keys_to_expire.count()} existing records to expire.")
        gold_delta_table.alias("history") \
            .merge(
                keys_to_expire.alias("updates"),
                "history.product_id = updates.product_id AND history.is_current = true"
            ) \
            .whenMatchedUpdate(set={
                "is_current": lit(False),
                "end_date": col("updates.update_timestamp")
            }) \
            .execute()

    if records_to_insert.count() > 0:
        print(f"Found {records_to_insert.count()} new, changed, or forced records to append as current.")

        new_current_records = records_to_insert \
            .select("product_id", "product_name", "price", "category", "update_timestamp") \
            .withColumn("is_current", lit(True)) \
            .withColumn("start_date", col("update_timestamp")) \
            .withColumn("end_date", lit(None).cast(TimestampType()))

        print("Appending new current records to Gold table.")
        new_current_records.write.format("delta") \
            .option("mergeSchema", "true") \
            .mode("append") \
            .save(gold_path)
    else:
        print("No product changes found. No updates to the Gold table.")

except Exception as e:
    print(f"Error in Gold SCD processing: {e}")
    import traceback
    traceback.print_exc()
    job.commit()
    sys.exit(1)

print("Glue job finished successfully.")
job.commit()

```

glue-demo Last modified on 10/6/2025, 7:57:56 PM

Script | Job details | **Runs** | Data quality | Schedules | Version Control

Job runs (1/8) info Last updated (UTC) October 6, 2025 at 15:05:02 [View details](#) [Stop job run](#) [Troubleshoot with AI](#) [Table View](#) [Card View](#)

Filter job runs by property

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
Failed	0	10/06/2025 20:14:08	10/06/2025 20:15:35	1 m 15 s	2 DPUs	G.1X	5.0
Failed	0	10/06/2025 19:59:11	10/06/2025 20:00:29	1 m 9 s	2 DPUs	G.1X	5.0
Failed	0	10/06/2025 19:49:15	10/06/2025 19:50:36	1 m 13 s	2 DPUs	G.1X	5.0
Succeeded	0	10/06/2025 19:44:49	10/06/2025 19:46:56	2 m	2 DPUs	G.1X	5.0

Run details **Input arguments (13)** **Logs** **Run insights** **Metrics** **Troubleshooting analysis - preview** **Spark UI**

Job name: glue-demo
 Job ID: j1-ec257be171fec5f8a1d4817c19c78fafc8279ee7b1ba6b0e584e42647639c956
 Run status: Succeeded
 Retry attempt number: 0
 Initial run: 0
 Trigger name: -

Start time (Local): 10/06/2025 19:44:49
 End time (Local): 10/06/2025 19:46:56
 Start-up time: 7 seconds
 Execution time: 2 minutes
 Security configuration: -

Glue version: 5.0
 Worker type: G.1X
 Max capacity: 2 DPUs
 Execution class: Standard
 Cloudwatch logs: [Output logs](#)
 Error logs: [Error logs](#)

Last modified on (Local): 10/06/2025 19:46:56
 Log group name: /aws-glue/jobs

Number of workers: 2
 Timeout: 480 minutes
 Usage profile: -

glue-demo Last modified on 10/6/2025, 7:57:56 PM

Script | **Job details** | Runs | Data quality | Schedules | Version Control

Libraries info

Python library path:

Dependent JARs path:

Referenced files path:

Additional Python modules path:

AWS recommends pinning library versions to ensure a consistent job environment.

Job parameters info

Key: Value - optional

--conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension [Remove](#)

--datalake-formats delta [Remove](#)

[Add new parameter](#)

You can add 48 more parameters.

```
--conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension --conf
spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog

--datalake-formats delta
```

Run producer.py:

- Silver Layer: The data is cleaned (casting data types, trimming strings), validated (e.g., price > 0), and de-duplicated to create a table representing the most current state of each product. This table is updated using a MERGE (upsert) operation.
 - Gold Layer: The final, business-aggregated table is created. This project implements a Slowly Changing Dimension (SCD) Type 2 table to track the complete history of product prices. Each time a product's price or details change, the old record is expired (by setting is_current = false and an end_date), and a new record is inserted as the current version.
4. Serverless Querying: The Gold Delta table in S3 is registered with the Glue Data Catalog and can be easily queried using standard SQL through Amazon Athena, providing a serverless analytics solution.

```
CREATE DATABASE IF NOT EXISTS medallion_db

CREATE EXTERNAL TABLE IF NOT EXISTS default.prices_history_gold ( product_id BIGINT, product_name
STRING, price DOUBLE, category STRING, update_timestamp TIMESTAMP, is_current BOOLEAN, start_date
TIMESTAMP, end_date TIMESTAMP ) STORED AS PARQUET LOCATION 's3://project-medallion-
ncpl/gold/prices_history/' TBLPROPERTIES ( 'parquet.compress'='SNAPPY', 'has_encrypted_data'='false'
)

SELECT * FROM prices_history_gold
ORDER BY product_id, start_date DESC

DESCRIBE prices_history_gold
DESCRIBE prices_history_gold
```

project-medallion x project-raw-nq x demo-lambda x CloudWatch [a] x Query editor [a] x Create workgro x CloudWatch [a] x Google Gemini x +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/workgroups/create

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udem... Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon Athena > Workgroups > Create workgroup

Workgroup description must be from 1-1024 characters. 1024 characters remaining.

Analytics engine - new [info](#)

Choose the type of engine

☒ **Athena SQL**
Athena SQL engine for interactive queries.

☐ **Apache Spark**
Highly-scalable serverless Spark engine.

Upgrade query engine

☒ **Automatic**
Let Athena choose when to upgrade your workgroup. [info](#)

☐ **Manual**
Manually choose an engine version now.

Authentication - new [info](#)

Authentication

☒ **AWS Identity and Access Management (IAM)**
Authenticate using IAM to centrally manage permissions that control which AWS resources users can access.

☐ **AWS IAM Identity Centre**
Authenticate with single sign-on using IAM Identity Centre (recommended to centrally manage access permissions for multiple AWS accounts). You can add users once the workgroup is created.

You cannot change the authentication after the workgroup is created.

Query result configuration [info](#)

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Gold +1.56%

23:58 06-10-2025

project-medallion x project-raw-nq x demo-lambda x CloudWatch [a] x Query editor [a] x Workgroups [A] x CloudWatch [a] x Google Gemini x +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/workgroups

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udem... Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon Athena > Workgroups

Amazon Athena <

Query editor tabs
Notebook editor
Notebook explorer

Jobs

Workflows
Powered by Step Functions

Administration

Workgroups
Capacity reservations
Data sources and catalogues

What's new [SH](#)

☐ Turn on compact mode

Workgroups (2) [info](#)

Use workgroups to separate users, teams, applications, workloads, and to set limits on amount of data for each query or the entire workgroup process. You can also view query-related metrics in Amazon CloudWatch.

Filter work groups

Name	Description	Analytics engine	Engine updates	Authentication	Created on	State
☐ Project	-	Athena engine version 3	Automatic	IAM	2025-10-06T23:44:20.217+0...	Turned On
☐ primary	-	Athena engine version 3	Automatic	IAM	2025-10-06T01:18:55.815+0...	Turned On

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Hot days ahead 25°C

23:58 06-10-2025

gold/prices_history/part-0 project-raw-ncpl - S3 buck Runs - Editor - AWS Glue CloudWatch | ap-south-1 Google Gemini

ap-south-1.console.aws.amazon.com/s3/buckets/project-raw-ncpl?region=ap-south-1&bucketType=general&prefix=processed/&showversions=false

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

glue

Amazon S3 Buckets project-raw-ncpl processed/

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight

AWS Marketplace for S3

processed/

Objects Properties

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	combined_data_3ea07710-234c-4cc4-9a91-3a7f9b930302.json	json	October 6, 2025, 23:12:15 (UTC+05:30)	115.0 B	Standard
☐	combined_data_49d1ca1b-a64b-4f1b-b4bc-fd5a5631ede7.json	json	October 6, 2025, 23:07:40 (UTC+05:30)	1.2 KB	Standard
☐	combined_data_d339c4c6-d10d-4147-b28f-be011b47e84b.json	json	October 6, 2025, 23:23:32 (UTC+05:30)	367.0 B	Standard

CloudShell Feedback

Thunderstorm w... In effect

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

gold/prices_history/part-0 project-raw-ncpl - S3 buck Runs - Editor - AWS Glue CloudWatch | ap-south-1 Google Gemini

ap-south-1.console.aws.amazon.com/s3/buckets/project-raw-ncpl?region=ap-south-1&bucketType=general&tab=objects

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

glue

Amazon S3 Buckets project-raw-ncpl

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight

AWS Marketplace for S3

project-raw-ncpl

Objects Properties Permissions Metrics Management Access Points

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	2025/	Folder	-	-	-
☐	processed/	Folder	-	-	-

CloudShell Feedback

Thunderstorm w... In effect

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

gold/prices_history/part-01 x project-raw-ncpl - S3 buck x Runs - Editor - AWS Glue x CloudWatch | ap-south-1 x Google Gemini x +

ap-south-1.console.aws.amazon.com/s3/buckets/project-raw-ncpl?region=ap-south-1&bucketType=general&prefix=2025/10/06/17/8&showversions=false

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

glue

Amazon S3 Buckets project-raw-ncpl 2025/ 10/ 06/ 17/

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight 11

AWS Marketplace for S3

17/

Objects Properties

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	KD5-S3-v74m-1-2025-10-06-17-36-314c20043-7e32-4ba6-b64b-24d11c85c931	-	October 6, 2025, 23:07:38 (UTC+05:30)	950.0 B	Standard
☐	KD5-S3-v74m-1-2025-10-06-17-41-09-62f61d2c-e2cc-4378-b024-4d9a593d0941	-	October 6, 2025, 23:12:14 (UTC+05:30)	89.0 B	Standard
☐	KD5-S3-v74m-1-2025-10-06-17-52-25-00339889-325f-43bb-a7d3-076b9e979461	-	October 6, 2025, 23:23:30 (UTC+05:30)	295.0 B	Standard

https://ap-south-1.console.aws.amazon.com/s3/object/project-raw-ncpl?region=ap-south-1...

Thunderstorm w... In effect

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

gold/prices_hist x project-raw-ncpl x demo-lambda | x CloudWatch | ap x Runs - Editor - x Runs - Editor - x CloudWatch | ap x Google Gemini x +

ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#logsV2:log-groups/log-group/\$252faws\$252fdemo-lambda\$log-events/2025\$252f...

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

CloudWatch

Log groups /aws/lambda/demo-lambda 2025/10/06/[SLATEST]J98628644bc74802b9c9f7eaf4fb1b4

CloudWatch

Favorites and recent

Dashboards

AI Operations New

Overview

Investigations

Configuration

Alarms 0 0 0 0

In alarm

All alarms New

Logs

Log groups

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

Metrics

All metrics New

Explorer

Streams

Application Signals (APM)

CloudShell Feedback

Top Stories Global trade cal...

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom UTC timezone Display

Timestamp	Message
No older events at this moment. Retry	
2025-10-06T17:53:38.425Z	INIT_START Runtime Version: python:3.13.463 Runtime Version ABI: arn:aws:lambda:ap-south-1:runtime:6638c930189402131f080807f99206155387f859c363c3d4808085642c87
2025-10-06T17:53:31.174Z	START RequestId: d3394c5-d38d-4347-b28f-beb13b07e94b Version: SLATEST
2025-10-06T17:53:31.533Z	Combined JSON written to s3://project-raw-ncpl/processed/combined_data_d3394c5-d38d-4347-b28f-beb13b07e94b.json
2025-10-06T17:53:31.828Z	Glue job triggered successfully: j_r_36d8997d8a3bce16251eeac4e08f736c47320f3683aace838ec023731a
2025-10-06T17:53:31.844Z	END RequestId: d3394c5-d38d-4347-b28f-beb13b07e94b
2025-10-06T17:53:31.844Z	REPORT RequestId: d3394c5-d38d-4347-b28f-beb13b07e94b Duration: 649.62 ms Billed Duration: 1215 ms Memory Size: 328 MB Max Memory Used: 98 MB Init Duration: 945.89 ms
No newer events at this moment. Auto retry paused. Resume	

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

gold/prices_hist x project-raw-ncpl x demo-lambda | x CloudWatch | ap: x Runs - Editor - / x Runs - Editor - / x CloudWatch | ap: x Google Gemini x +

ap-south-1.console.aws.amazon.com/gluestudio/home?region=ap-south-1#/editor/job/glue-demo/runs

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

AWS Glue

Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)

Data Catalog
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings

Data Integration and ETL
Legacy pages

What's New
Documentation
AWS Marketplace

Enable compact mode
Enable new navigation

glue-demo

Last modified on 10/6/2025, 11:06:19 PM Actions Save Run

Script Job details **Runs** Data quality Schedules Version Control

Job runs (1/19) info

Filter job runs by property Last updated 8/12/2025 at 18:22:18 View details Stop job run Troubleshoot with AI Table View Card View

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPU)	Worker type	Glue version
Succeeded	0	10/06/2025 23:23:31	10/06/2025 23:25:45	1 m 59 s	2 DPUs	G.1X	5.0
Succeeded	0	10/06/2025 23:12:14	10/06/2025 23:14:11	1 m 50 s	2 DPUs	G.1X	5.0
Succeeded	0	10/06/2025 23:07:39	10/06/2025 23:09:21	1 m 33 s	2 DPUs	G.1X	5.0
Failed	0	10/06/2025 23:02:44	10/06/2025 23:03:16	15 s	2 DPUs	G.1X	5.0

Run details

Input arguments (13) Logs Run insights Metrics Troubleshooting analysis - preview Spark UI

Job name	Start time (Local)	Glue version	Last modified on (Local)
glue-demo	10/06/2025 23:23:31	5.0	10/06/2025 23:25:45
Id	End time (Local)	Worker type	Log group name
j1-366b997d5a3b5e316251ee35c4e0bf716cd73205f3603a3ace	10/06/2025 23:25:45	G.1X	/aws-glue/jobs
as10ec023f31a			
Run status	Start-up time	Max capacity	Number of workers
Succeeded	14 seconds	2 DPUs	2
Retry attempt number	Execution time	Execution class	Timeout
Initial run	1 minute 59 seconds	Standard	480 minutes
Trigger name	Security configuration	Cloudwatch logs	Usage profile
-	-	Output logs	-
-	-	Error logs	-

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Top Stories Global trade cal...

project-medallion- x project-raw-ncpl x demo-lambda | x CloudWatch | ap: x Runs - Editor - / x Runs - Editor - / x CloudWatch | ap: x Google Gemini x +

ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-south-1&bucketType=general&tab=objects

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon S3 Buckets project-medallion-ncpl

project-medallion-ncpl info

Objects Properties Permissions Metrics Management Access Points

Objects (7)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
bronze_folders	-	October 6, 2025, 23:08:45 (UTC+05:30)	-	0 B Standard
bronze/	Folder	-	-	-
gold_folders	-	October 6, 2025, 23:09:06 (UTC+05:30)	-	0 B Standard
gold/	Folder	-	-	-
parfiles/	Folder	-	-	-
silver_folders	-	October 6, 2025, 23:09:01 (UTC+05:30)	-	0 B Standard
silver/	Folder	-	-	-

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

JPY/INR -1.86%

project-medallion x project-rw-ncpl x demo-lambda | x CloudWatch | ap x Runs - Editor - / x Runs - Editor - / x CloudWatch | ap x Google Gemini x +

ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-south-1&bucketType=general&prefix=gold/prices_history/&showversions=false

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon S3 > Buckets > project-medallion-ncpl > gold/ > prices_history/

prices_history/

Copy S3 URI

Objects Properties

Objects (6)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	_delta_log_3folder5	-	October 6, 2025, 23:09:06 (UTC+05:30)	0 B	Standard
☐	_delta_log/	Folder	-	-	-
☐	part-00000-a465a7c0-5874-40f2-8d1b-3cc8504199f1-c000.snappy.parquet	parquet	October 6, 2025, 23:25:26 (UTC+05:30)	2.9 KB	Standard
☐	part-00000-ad797f5e-12e9-436d-beca-f5b014d89e65-c000.snappy.parquet	parquet	October 6, 2025, 23:09:07 (UTC+05:30)	2.8 KB	Standard
☐	part-00000-c82a015a-da5f-4561-9f4e-5fb57f5f6646-c000.snappy.parquet	parquet	October 6, 2025, 23:13:58 (UTC+05:30)	2.3 KB	Standard
☐	part-00000-fb2f690d-fd0c-4d59-b60a-e32fe56784d1-c000.snappy.parquet	parquet	October 6, 2025, 23:25:31 (UTC+05:30)	2.4 KB	Standard

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

JPY/INR -1.86%

23:56 06-10-2025

project-medallion x project-rw-ncpl x demo-lambda | x CloudWatch | ap x Runs - Editor - / x Runs - Editor - / x CloudWatch | ap x Google Gemini x +

ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-south-1&bucketType=general&prefix=silver/prices/&showversions=false

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon S3 > Buckets > project-medallion-ncpl > silver/ > prices/

prices/

Copy S3 URI

Objects Properties

Objects (9)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	_delta_log_3folder5	-	October 6, 2025, 23:09:02 (UTC+05:30)	0 B	Standard
☐	_delta_log/	Folder	-	-	-
☐	part-00000-3d11da1-f503-4b0f-b30f-4b7252a4d8ba-c000.snappy.parquet	parquet	October 6, 2025, 23:25:13 (UTC+05:30)	1.8 KB	Standard
☐	part-00000-6a6335c6-1a05-475d-ad6e-1d82975e1ec4-c000.snappy.parquet	parquet	October 6, 2025, 23:09:02 (UTC+05:30)	2.2 KB	Standard
☐	part-00000-a7e1831b-8a17-4c7b-ba8e-80e396f17b2f-c000.snappy.parquet	parquet	October 6, 2025, 23:13:47 (UTC+05:30)	792.0 B	Standard
☐	part-00001-daeae887-836f-4f33-abdf-093975e6735c-c000.snappy.parquet	parquet	October 6, 2025, 23:25:13 (UTC+05:30)	1.9 KB	Standard
☐	part-00002-cfcaa42c-5335-4345-b6d9-a1c822276101-c000.snappy.parquet	parquet	October 6, 2025, 23:25:13 (UTC+05:30)	1.9 KB	Standard
☐	part-00003-9e58d996-8019-415c-87a2-11cf83f1c05d-c000.snappy.parquet	parquet	October 6, 2025, 23:25:13 (UTC+05:30)	1.9 KB	Standard
☐	part-00001-c06ec976-5a32-4d12-b6cc-edadfc8842676-c000.snappy.parquet	parquet	October 6, 2025, 23:13:47 (UTC+05:30)	1.8 KB	Standard

https://ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-s...

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

JPY/INR -1.86%

23:56 06-10-2025

project-medallion x project-law-ncpl x demo-lambda | x CloudWatch | ap x Runs - Editor - / x Runs - Editor - / x CloudWatch | ap x Google Gemini x +

ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-south-1&bucketType=general&prefix=bronze/prices/&showversions=false

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udem... Notes Black and White Cle... All Bookmarks

Search [Alt+S]

Account ID: 4876-1574-3519 Yadwarth

Amazon S3 Buckets project-medallion-ncpl bronze/ prices/

prices/ Copy S3 URI

Objects Properties

Objects (5)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
.delta_log/	Folder	October 6, 2025, 23:08:45 (UTC+05:30)	0 B	Standard
part-00000-0938870c-4e13-4bc7-8f6c-d729b37ee90e-c000.snappy.parquet	parquet	October 6, 2025, 23:08:48 (UTC+05:30)	1.8 KB	Standard
part-00000-118362d7-9c28-4b5a-b5eb-af45043da27d-c000.snappy.parquet	parquet	October 6, 2025, 23:13:26 (UTC+05:30)	1.5 KB	Standard
part-00000-1ade392e-6c45-44a9-925d-7876a1209515-c000.snappy.parquet	parquet	October 6, 2025, 23:24:52 (UTC+05:30)	1.6 KB	Standard

https://ap-south-1.console.aws.amazon.com/s3/buckets/project-medallion-ncpl?region=ap-s... © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

JPY/INR 1.66% ENG IN 23:56 06-10-2025

project-medallion-rx x project-raw-ncpl - S x demo-lambda | Fun x CloudWatch | ap-sou x Query editor tabs | A x CloudWatch | ap-sou x Google Gemini x +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/query-editor/history/47dd350d-684e-4402-bc58-4c2e743bc1bc

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udem... Notes Black and White Cle... All Bookmarks

Search [Alt+S]

Amazon Athena > Query editor tabs

Amazon Athena

Query editor tabs

Notebook editor

Notebook explorer

Jobs

Workflows

Powered by Step Functions

Administration

Workgroups

Capacity reservations

Data sources and catalogues

What's new

Turn on compact mode

Editor

Recent queries

Saved queries

Settings

Workgroup: Project

Data

Data source: AwsDataCatalog

Catalogue: None

Database: default

Tables and views

Filter tables and views

Tables (1)

prices_history_gold

Views (0)

Query 2

```
1 CREATE DATABASE IF NOT EXISTS medallion_db
```

Run again Explain (2) Cancel Clear Create

Query results

Query status

Completed

Time in queue: 36 ms Run time: 514 ms Data scanned: -

query successful.

Reuse query results up to 60 minutes ago

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Gold +1.56%

23:58 06-10-2025

project-medallion-rx x project-raw-ncpl - S x demo-lambda | Fun x CloudWatch | ap-sou x Query editor tabs | A x CloudWatch | ap-sou x Google Gemini x +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/query-editor/history/7eb0d644-cd87-4ee9-9f09-5790884b7f20

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udem... Notes Black and White Cle... All Bookmarks

Search [Alt+S]

Amazon Athena > Query editor tabs

Amazon Athena

Query editor tabs

Notebook editor

Notebook explorer

Jobs

Workflows

Powered by Step Functions

Administration

Workgroups

Capacity reservations

Data sources and catalogues

What's new

Turn on compact mode

Editor

Recent queries

Saved queries

Settings

Workgroup: Project

Data

Data source: AwsDataCatalog

Catalogue: None

Database: default

Tables and views

Filter tables and views

Tables (1)

prices_history_gold

Views (0)

Query 2

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS default.prices_history_gold ( product_id BIGINT, product_name STRING, price DOUBLE, category STRING, update_timestamp TIMESTAMP, is_current BOOLEAN, start_date TIMESTAMP, end_date TIMESTAMP ) STORED AS PARQUET LOCATION 's3://project-medallion-ncpl/gold/prices_history/' TBLPROPERTIES ('parquet.compress'='SNAPPY', 'has_encrypted_data'='false')
```

Run again Explain (2) Cancel Clear Create

Query results

Query status

Completed

Time in queue: 39 ms Run time: 434 ms Data scanned: -

query successful.

Reuse query results up to 60 minutes ago

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Air, Moderate Tomorrow

00:00 07-10-2025

project-medall... project-law-nq... demo-lambda... CloudWatch... Query editor... Query editor... CloudWatch... Google Gemini... +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/query-editor/history/9478bef4-d436-4860-a199-c1a40f3ed6a5

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon Athena > Query editor tabs

Amazon Athena

Query editor tabs

Notebook editor

Notebook explorer

Jobs

Workflows

Powered by Step Functions

Administration

Workgroups

Capacity reservations

Data sources and catalogues

What's new

Turn on compact mode

Data

Data source

AwsDataCatalog

Catalogue

None

Database

default

Tables and views

Create

Filter tables and views

Tables (1)

prices_history_gold

product_id bigint

product_name string

price double

category string

update_timestamp timestamp

is_current boolean

start_date timestamp

end_date timestamp

Views (0)

Query 2 Query 4 Query 3 Query 5 Query 6

1 DESCRIBE prices_history_gold

SQL Ln 1, Col 1

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results

Query status

Completed

Time in queue: 131 ms Run time: 806 ms Data scanned: -

product_id bigint

product_name string

price double

category string

update_timestamp timestamp

is_current boolean

start_date timestamp

end_date timestamp

CloudShell Feedback

25°C Mostly cloudy

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

project-medall... project-law-nq... demo-lambda... CloudWatch... Query editor... Query editor... CloudWatch... Google Gemini... +

ap-south-1.console.aws.amazon.com/athena/home?region=ap-south-1#/query-editor/history/0981eb19-71b3-47d1-8ee5-183e2fc63756

OCI eyuu Python Windup Gmail LinkedIn Perplexity Gemini https://chat.openai... YouTube Confluence My learning | Udemy Notes Black and White Cle... All Bookmarks

Search [Alt+S]

EC2 S3 DynamoDB Aurora and RDS Amazon Redshift

Amazon Athena > Query editor tabs

Amazon Athena

Editor Recent queries Saved queries Settings

Workgroup Project

Query editor tabs

Notebook editor

Notebook explorer

Jobs

Workflows

Powered by Step Functions

Administration

Workgroups

Capacity reservations

Data sources and catalogues

What's new

Turn on compact mode

Data

Data source

AwsDataCatalog

Catalogue

None

Database

default

Tables and views

Create

Filter tables and views

Tables (1)

prices_history_gold

product_id bigint

product_name string

price double

category string

update_timestamp timestamp

is_current boolean

start_date timestamp

end_date timestamp

Views (0)

Query 2 Query 4 Query 3 Query 5

1 SHOW TABLES IN default

SQL Ln 1, Col 1

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results

Query status

Completed

Time in queue: 35 ms Run time: 311 ms Data scanned: -

prices_history_gold

CloudShell Feedback

Trending videos

Avatar: Fire and...

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Amazon Athena Query editor interface. The left sidebar shows navigation options: Query editor tabs, Notebook editor, Notebook explorer, Jobs, Workflows, Administration, Workgroups, Capacity reservations, and Data sources and catalogues. The main area displays the 'Data' tab with a table named 'prices_history_gold' and its schema. The 'Query' tab shows a SQL query: `SELECT * FROM prices_history_gold ORDER BY product_id, start_date DESC`. The 'Query results' tab shows the query status as 'Completed' with a time in queue of 61 ms and a run time of 517 ms. The 'Results (24)' tab displays a table with 24 rows of product data.

#	product_id	product_name	price	category	update_timestamp	is_current	start_date	end_date
1	101	Laptop Pro 15	1200.99	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
2	101	Laptop Pro 15	1299.99	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
3	101	Laptop Pro 15	1299.99	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
4	102	Wireless Mouse	20.5	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
5	102	Wireless Mouse	25.5	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
6	102	Wireless Mouse	25.5	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
7	103	Mechanical Keyboard	70.99	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
8	103	Mechanical Keyboard	75.99	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
9	103	Mechanical Keyboard	75.99	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
10	104	Organic Coffee Beans	18.75	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
11	104	Organic Coffee Beans	18.75	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
12	105	Green Tea - 100 Bags	9.99	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
13	105	Green Tea - 100 Bags	9.99	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
14	106	Running Shoes	89.95	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
15	106	Running Shoes	89.95	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
16	107	Cotton T-Shirt	15.0	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	

Amazon Athena Query editor interface showing the 'Results (24)' tab. The table displays 24 rows of product data, including product_id, product_name, price, category, update_timestamp, is_current, start_date, and end_date. The interface includes a search bar for results and a 'Download results CSV' button.

#	product_id	product_name	price	category	update_timestamp	is_current	start_date	end_date
1	101	Laptop Pro 15	1200.99	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
2	101	Laptop Pro 15	1299.99	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
3	101	Laptop Pro 15	1299.99	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
4	102	Wireless Mouse	20.5	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
5	102	Wireless Mouse	25.5	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
6	102	Wireless Mouse	25.5	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
7	103	Mechanical Keyboard	70.99	Electronics	2025-10-06 17:55:29.903	true	2025-10-06 17:55:29.903	
8	103	Mechanical Keyboard	75.99	Electronics	2025-10-06 17:39:05.864	false	2025-10-06 17:39:05.864	2025-10-06 17:55:21.062
9	103	Mechanical Keyboard	75.99	Electronics	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
10	104	Organic Coffee Beans	18.75	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
11	104	Organic Coffee Beans	18.75	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
12	105	Green Tea - 100 Bags	9.99	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
13	105	Green Tea - 100 Bags	9.99	Groceries	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
14	106	Running Shoes	89.95	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
15	106	Running Shoes	89.95	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	
16	107	Cotton T-Shirt	15.0	Apparel	2025-10-06 17:39:05.864	true	2025-10-06 17:39:05.864	

Amazon Athena query result download

Download query result: b9b6fdbb-a31e-4d34-b2d4-3d324bcb7d31

Your query result download was successful. You may safely close the window.

Close

b9b6fdbb-a31e-4d34-b2d4-3d324bcb7d31.csv
2,305 B • Done

part-00000-fb2f690d-fd0c-4d59-b60a-e32fe56784d1-c000.snappy.parquet
2,453 B • 27 minutes ago

part-00000-c82e015a-da5f-4561-9f4e-5fb57f5f6e4b-c000.snappy (1).parquet
2,344 B • 28 minutes ago

part-00000-ad797f5e-12e9-43fd-beca-f5b01dd89e65-c000.snappy (1).parquet
2,902 B • 29 minutes ago

part-00000-a465a7c0-0f74-40f2-8d18-3cc8504199f1-c000.snappy.parquet
3,009 B • 34 minutes ago

product_id	product_name	price	category	update_time	is_current	start_date	end_date
101	Laptop Prc	1200.99	Electronics	55:29.9	TRUE	55:29.9	
101	Laptop Prc	1299.99	Electronics	39:05.9	FALSE	39:05.9	55:21.1
101	Laptop Prc	1299.99	Electronics	39:05.9	TRUE	39:05.9	
102	Wireless N	20.5	Electronics	55:29.9	TRUE	55:29.9	
102	Wireless N	25.5	Electronics	39:05.9	FALSE	39:05.9	55:21.1
102	Wireless N	25.5	Electronics	39:05.9	TRUE	39:05.9	
103	Mechanics	70.99	Electronics	55:29.9	TRUE	55:29.9	
103	Mechanics	75.99	Electronics	39:05.9	FALSE	39:05.9	55:21.1
103	Mechanics	75.99	Electronics	39:05.9	TRUE	39:05.9	
104	Organic Cc	18.75	Groceries	39:05.9	TRUE	39:05.9	
104	Organic Cc	18.75	Groceries	39:05.9	TRUE	39:05.9	
105	Green Tea	9.99	Groceries	39:05.9	TRUE	39:05.9	
105	Green Tea	9.99	Groceries	39:05.9	TRUE	39:05.9	
106	Running Sh	89.95	Apparel	39:05.9	TRUE	39:05.9	
106	Running Sh	89.95	Apparel	39:05.9	TRUE	39:05.9	
107	Cotton T-S	15	Apparel	39:05.9	TRUE	39:05.9	
107	Cotton T-S	15	Apparel	39:05.9	TRUE	39:05.9	
108	The Great	12.49	Books	39:05.9	TRUE	39:05.9	
108	The Great	12.49	Books	39:05.9	TRUE	39:05.9	
109	Learning P	39.99	Books	39:05.9	TRUE	39:05.9	
109	Learning P	39.99	Books	39:05.9	TRUE	39:05.9	
110	Yoga Mat	29.99	Sports	39:05.9	TRUE	39:05.9	
110	Yoga Mat	29.99	Sports	39:05.9	TRUE	39:05.9	
111	Iphone 13	120	Electronics	43:56.2	TRUE	43:56.2	

Challenges and Solutions

During development, a recurring error in the AWS Glue job was `DELTA_FAILED_TO_MERGE_FIELDS`.

- Problem: This error occurred during the MERGE operation into the Delta tables. It was caused by a schema mismatch between the incoming data (from the JSON files) and the existing Delta table, specifically with the price column.
- Investigation: Analysis showed that while the data was numeric, PySpark's schema inference was not consistently matching the Delta table's schema (e.g., inferring long vs. double).
- Solution: The issue was resolved by implementing a multi-step, robust solution within the PySpark script:
 1. Explicit Schema Definition: A rigid schema (`bronze_schema`) was defined, explicitly casting the price column to `DoubleType` upon reading the raw JSON data.
 2. Schema Evolution: Spark configurations were set to allow for schema merging (`spark.databricks.delta.schema.autoMerge.enabled`).
 3. Error Handling & Overwrite: A try-except block was added. If the standard append with schema merge fails, the script now has logic to read the existing data, cast it to the new corrected schema, union it with the new data, and overwrite the table, thereby healing the schema issue for all future runs.

This solution not only fixed the immediate error but also made the pipeline more resilient to future schema variations.